DEVELOPER HANDBOOK

Working on Eswachh

How the code is arranged, where every switch lives, what to edit for the changes you are most likely to be asked for, and the mistakes this codebase has already made so you do not repeat them.

STACK	Laravel 12 · PHP 8.2+ · Vue 3.5 · TypeScript 5.9 · Vite 7 · Pinia · TanStack Query · Tailwind 4 · MySQL
SUITE	395 tests, 1102 assertions, ~65s
COMPANION	admin-guide.pdf, for what the system does and why
REVISED	15 August 2026

CONTENTS

What is in this handbook

SECTION 01

Getting it running

1.1

```
# everything: composer, .env, key, migrate, npm, build
composer setup

# the price list, message templates, policies, a first admin
php artisan db:seed

# server + queue + log tail + vite, all four at once
composer dev
```

`composer dev` runs `php artisan serve`, the queue listener, `pail` and `npm run dev` together and kills all four when any one stops. Use it rather than four terminals.

What must be in .env

KEY	WHY
`APP_TIMEZONE=Asia/Kolkata`	**Not optional.** v1 ran on it. On UTC, imported timestamps are mislabelled by five and a half hours and date filters silently find nothing.
`DB_CONNECTION=mysql`	The importer reads a v1 MySQL database directly.
`SESSION_DRIVER=database`	Sanctum SPA cookie auth.
`QUEUE_CONNECTION=database`	Messaging is queued.

ONE COMMAND TO ANSWER "IS IT WIRED UP?"

`php artisan eswachh:check-integrations` reports whether payments and messaging are configured and, when they are not, exactly what is stopping them. Add `--ping` to prove the gateway key actually works.

Importing v1 data

app/Console/Commands/ImportLegacyData.php

`php artisan eswachh:import` brings across customers, vehicles, subscriptions and payments, links every customer to a login that still accepts their old password, and issues invoice numbers for payments that had none. Five v1 orders cannot come across: their customers were deleted in v1, so there is nobody to attach them to.

SECTION 02

How the code is arranged

2.1 Everything is filed by **who it is for**, not by what kind of file it is. The same four names appear on both sides of the wire.

AUDIENCE	BACKEND	FRONT END
The office	`app/Http/Controllers/Api/V1/Admin`	`resources/js/admin`
A customer	`.../Api/V1/Portal`	`resources/js/portal`
A visitor	`.../Api/V1/Site`	`resources/js/site`
More than one	`.../Api/V1/Shared`	`resources/js/shared`

Why not group by resource

A `SubscriptionController` holding everything anyone can do to a subscription hides the only question that matters here: *who is asking*. A customer holds `view.subscription` so they can look at their own plan. An ability cannot say "their own"; only a query can. Filing by audience puts that question in the folder name.

> **SHARED IS THE LOAD-BEARING ONE**
>
> Those controllers are reached by both the office and a customer, so each carries a row-level check inside it (`RestrictsToOwnRecords`). If one is ever moved into `Admin`, that check starts to look redundant to whoever edits it next — which is exactly how a customer ends up reading the whole branch's payments.

2.2

What is not in that table

FOLDER	HOLDS
`app/Domain/*`	The business itself: pricing, billing, messaging, the daily round, cloth ledger, complaints, alerts, phone codes. No HTTP, no request objects. A controller validates and delegates; the rules read the same whether a person, a scheduled job or a test called them.
`app/Support/*`	Plumbing that is not a business rule: `Tenancy` , `Access` , `Http` (sorting whitelists, ownership), `Settings` , `Numbering` , `Content` (sanitising), `Masters` , `Legacy` , `Database` , `Preferences` .
`app/Models`	One flat folder on purpose. Models are referenced from everywhere, and a `Subscription` filed under `Admin` would be a lie.

Adding a route

1. Decide who it is for. That is the folder.
2. If the answer is "both the office and a customer", it is `Shared` , and it needs an ownership check — `restrictToOwnRecords()` for a list, `ownsRecord()` for a single row, returning **404 and not 403**, because refusing must not confirm that the record exists.

3. Abilities go on the route or at the top of the method. They answer *may this person use this feature*, never *whose rows are these*.

```
routes/api.php — everything behind auth
routes/api_public.php — quotes, signup, renewal lookup, catalogue, blog
routes/web.php — which blade shell serves which URL
routes/console.php — the schedule
```

SECTION 03

Sector scoping

3.1

`app/Support/Tenancy/SectorContext.php` · `app/Models/Concerns/ScopedToSectors.php`

A **sector is the territory**, and it is the whole of tenancy. Staff are assigned sectors through `user_sector`; a customer sits in one sector; a record is visible when those two meet.

```
users ─< user_sector >─ sectors ─< customers ─< subscriptions ─< service_logs
                                          ∴ vehicles, complaints, cloth_entries
```

One rule: **a customer is visible to whoever covers the sector the customer sits in.** Nothing is copied onto the customer, so reassigning a sector is one pivot row and takes effect on the next request.

```php
SectorContext::currentSectorIds()     // null for an admin, their ids otherwise
SectorContext::currentCustomerId()    // set only when the viewer IS a customer
SectorContext::isRestricted()         // false only for super_admin
SectorContext::withoutScope(fn () => ...)   // masters, cross-sector checks
SectorContext::forSectors($ids, fn () => ...) // jobs, importers, tests
SectorContext::forget($userId)        // after their assignment changes
SectorContext::reset()                // tests, in setUp and tearDown
```

> **CURRENTSECTORIDS ANSWERS NULL INSIDE WITHOUTSCOPE**
>
> Null means "ask nothing of this query", which is right for the scope and wrong everywhere else. Called inside an unscoped block it turned every franchise user's own territory into an empty list on `/me`. Resolve it *before* lifting the scope.

What each model declares

Put `use ScopedToSectors;` on a model and it reduces to "which customers may this viewer see". Three tables do not and say so in `sectorScope()`:

TABLE	SCOPED BY
payments	A **stamped** `sector_id`, written at capture and never recomputed. A payment records something that happened, so who took the money does not change when territory is rearranged — hand a sector over and the new holder gets the customers and the plans, but last year's figures do not change shape.
attendances	`cleaner_id` against the staff covering the viewer's sectors — the same pivot, asked from the other end.
sectors	Their own id: you see the ones you are assigned.

> **404, NOT 403**
>
> A record in another sector must produce the same response as one that never existed. A 403 confirms there is something there to be refused, which is itself a leak — it tells a franchise user that a car number belongs to somebody, somewhere.

3.2

Three rules that are easy to get wrong

- **Fail closed.** Covering nothing returns nothing, never everything. That is the difference between a quiet screen and a leak.
- **A customer with no `sector_id` belongs to nobody** until one is given. SQL `IN` never matches null, so the rule needs no special case — but `sector_id` is therefore *required* when the office creates a customer, or it silently makes an invisible one.
- **A customer is never in the pivot.** Their territory comes from their address; an assignment would let them see their neighbours. The scope asks `currentCustomerId()` first and narrows to their own records instead.

Assigning territory

On the **person**, under People — that is the moment it matters, because an account created without a sector signs in to empty screens. The Sectors screen reports who covers each one but does not edit it.

Nobody hands out territory they do not hold: a franchise owner adding a cleaner can only give them sectors from their own, or assigning a colleague elsewhere would be a way to read another franchise's customers through them.

When to use withoutScope

Only for things that are genuinely shared, and always with a comment saying why: **masters** (every franchise sells from the same price list, and this is why a quote works on the public site with nobody signed in), and **uniqueness of a car number**, since a car on another sector's books is still taken.

> The scope's own subqueries use the query builder rather than Eloquent, deliberately: they run inside the global scope that would otherwise call them again, and they must see soft-deleted rows so withdrawing a society does not hide its customers from the person still servicing them.

What this replaced

A `branch_id` copied onto the customer and onto everything of theirs. That column was a cache of where somebody lived, and it went stale: a sector was pointed at another franchise and two hundred customers stayed in the old owner's books while the sector claimed otherwise, with nothing anywhere reporting the disagreement. The columns are still on the tables, nullable and unread, until the model has run long enough to be trusted.

SECTION 04

Roles and abilities

4.1

`app/Enums/UserRole.php` · `app/Support/Access/Abilities.php` · `app/Models/CustomRole.php`

Four roles, declared in code rather than in tables: `super_admin`, `franchise_owner`, `cleaner`, `customer`. Abilities are named `{verb}.{resource}` and listed on the enum. `super_admin` is granted everything by a `Gate::before` hook, which keeps the table readable.

> WHY IN CODE
>
> v1 stored roles and permissions in tables and then cached the lookup — a cache that shadowed the relation and left newly created users with no roles at all until somebody cleared it by hand. Abilities declared in code cannot drift out of sync with themselves, cannot be cached wrongly, and are visible in review.

Adding an ability

1. Add the string to the relevant arm of `UserRole::abilities()`.
2. Add it to `Abilities` so the roles screen can offer it.
3. Check it on the route or at the top of the method.
4. If it is a screen, add it to the `navigation` array in `resources/js/admin/AppLayout.vue` and to the route's `meta.ability`.

> NEVER NAME AN ABILITY "OWN"
>
> An ability answers *may this person use this feature*. Which rows they see is a separate question that only a query can answer. Naming one `view.own.complaint` implied otherwise and left routes checking `view.complaint` locking customers out of their own complaints entirely.

Custom roles

DB-backed, and each **refines** one built-in role: it starts from that role's abilities and may only narrow or pick from within them. A custom role cannot grant cross-branch sight; there is no way to express it. Managing roles is gated on *being* `super_admin`, not on an ability, so that a role cannot be used to grant the power to rewrite roles.

SECTION 05

Money

5.1

app/Domain/Pricing/PriceBook.php · Quote.php · QuoteLine.php

```
( category + package + service_type + society surcharge ) × months
    − duration discount
    + cloth bundle
```

Every master is looked up fresh on each call. A quote is a statement about the price list *as it stands right now*, and must never be assembled from ids the caller has already resolved to amounts. A master that was asked for and cannot be sold produces a warning rather than being skipped — skipping it silently under-prices the plan by whatever it used to charge, and nobody would notice. Withdrawn and never-existed are told apart, because they need different fixes.

RULE ONE · NO AMOUNT EVER COMES FROM A REQUEST

Every price is worked out by `PriceBook`, on the server, at the moment it is charged. The figure the browser shows is a quote and is thrown away. v1 read `final_price` straight from the form and validated only `min:1` — a customer could have paid ₹1 for any plan.

There is a test for this on every paying route. If you add one, add the test: post a wrong `amount_paise` and assert the stored amount ignored it.

RULE TWO · NOTHING IS MARKED PAID EXCEPT A VERIFIED CALLBACK

Signup, renewal, top-up and add-a-car all open a payment and stop. `Domain\Billing\RecordPayment` moves it, and only after the Razorpay signature checks out *and* the gateway confirms what it holds.

5.2

The billing pieces

CLASS	DOES
`StartPayment`	Opens a payment row and a gateway order, before the window appears, so an abandoned checkout still leaves a record.
`RecordPayment`	The only thing that marks anything paid. Idempotent.
`RazorpaySignature`	HMAC verification, constant-time.
`RazorpayGateway`	The HTTP client. Reports itself unavailable unless enabled and keyed.
`InvoiceNumber`	Issues a numbered invoice per captured payment, prefix from settings.

Paise, everywhere

All money is an **integer number of paise**, named `*_paise`. Never a float, never a decimal column, no exceptions. Format for display at the edge only.

Repricing on renewal

> `app/Http/Controllers/Api/V1/Shared/RenewalController.php`

A customer renewing from their account may change package, service type and duration. The controller reprices and writes the new amount *before* opening the payment. Without that step a customer switching one month to six would be charged for one.

SECTION 06

Messaging

6.1

app/Domain/Messaging/Messenger.php · app/Enums/MessagePurpose.php · database/seeders/MessageTemplateSeeder.php

The delivery gate

```
public function deliveryEnabled(): bool
{
    // Never during tests, whatever the configuration says.
    if (app()->runningUnitTests()) {
        return false;
    }

    return app()->isProduction() && (bool) config('services.whatsapp.enabled');
}
```

THIS IS THE LINE THAT MATTERS

A message leaves the building only when the app is in production *and* `WHATSAPP_ENABLED=true`. Everywhere else it is written to the `messages` table exactly as it would have been sent, together with a `suppressionReason()` saying why it was held. Demonstrations against real data cannot message a real customer.

Do not add a bypass for "just this one". If you need to see delivery work, point a staging copy at a test number.

Wording lives in the database

Templates are seeded with the exact wording from the requirements document and edited in the office. A template that is missing or switched off sends **nothing**, rather than falling back to wording nobody has approved. Both the nightly jobs and hand-sent messages read the same row.

The dedupe key

`MessagePurpose` values are load-bearing: the purpose is half of the key that stops a customer being messaged twice about the same thing. **Renaming a case silently allows a duplicate.**

`isPerEvent()` splits the two behaviours. A renewal reminder is one per customer per day. A "cleaning done" is per event, because a household with two cars must get one for each.

SECTION 07

Every flag and switch

7.1 Three kinds. Environment variables are for credentials and things that differ per server. Settings are for the business. Hard-coded conditions are for what the business asked to hide.

Environment · .env

KEY	DEFAULT	READ BY
RAZORPAY_ENABLED	FALSE	RazorpayGateway::available()
RAZORPAY_KEY / _SECRET	empty	config/services.php
WHATSAPP_ENABLED	FALSE	Messenger::deliveryEnabled()
MSG91_AUTH_KEY	empty	Messenger
MSG91_WHATSAPP_NUMBER	empty	Messenger
APP_TIMEZONE	Asia/Kolkata	Everything. See section 13.

Settings · Settings screen, or `SiteSettings::put()`

> **app/Support/Settings/SiteSettings.php** — the whitelist of keys

KEY	DEFAULT	EFFECT
cloth_service_enabled	0	Hides the top-up page, the cloth screens and the cloth choice on the forms. Touches no balance or ledger entry.
lock_plan_edits_to_admin	0	Franchise owners cannot change a plan. Never restricts an administrator.
seo_index	1	Off writes `noindex`. Switch it off on staging.
cloth_low_threshold	5	The warning list and the low-balance message.
renewal_grace_days	7	Days overdue before a plan is put on hold.
admin_notify_phone	empty	Where "a new car needs a cleaner" goes. v1 had this number written into two controllers.
invoice_prefix	ESW	Invoice numbering.
seo_title / _description / _keywords / _share_image	set	Rendered server-side into `site.blade.php`.
privacy_policy / terms / refund_policy	written	Rich text, sanitised on write.
business_name, legal_name, gstin, address, contact_*, whatsapp_number, office_hours, invoice_footer	various	Presentation.

7.2

Hidden in code

WHAT	WHERE	TO RESTORE
Renew & Subscribe hidden from a signed-in customer	`resources/js/site/PublicLayout.vue` → `showVisitorActions`	Change the computed to `true`. Both the wide header and the narrow drawer read it.
Cloth top-up not in the public header	`resources/js/site/PublicLayout.vue` → the `links` array	Add `{ name: 'Cloths', to: { name: 'cloth-top-up' } }`. The route and page are untouched.
Car number editable by administrators only	`.../Admin/SubscriptionEditController.php` → `actorMayRenumber()`	Deliberate, from the requirements document. A refused change sets `$registrationRefused` so the response says so rather than staying silent.
Paid amount never editable	`.../Admin/SubscriptionEditController.php` → `applyPaymentChanges()`	Method, reference, paid-at and notes are editable; the amount is not. Do not add it.

Adding a setting

1. Add the key to `SiteSettings::definitions()` with a label, group, rules and default. Mark it `'boolean' => true`, `'rich' => true` or `'long' => true` as needed — the settings screen renders the right control from that.
2. Read it with `SiteSettings::get('your_key')`.
3. Nothing else. The cache rebuilds itself when the definition count changes, so a new setting does not need a cache clear to appear.

> **Nothing secret goes in settings.** Gateway keys and API tokens stay in `.env`, where they are not inside a database dump an administrator can download from the Backups screen.

SECTION 08

Change this, edit that

8.1

YOU ARE ASKED TO…	GO HERE
Change a price	Masters screen. Nothing in code. Prices are `*_paise` integers in the masters tables.
Change how a price is assembled	`app/Domain/Pricing/PriceBook.php — and add a test, because historic amounts must still reproduce.`
Change message wording	Messages screen. To change the *seed*: `database/seeders/MessageTemplateSeeder.php`.
Add a new message	A case on `MessagePurpose` (+ `label()`, and `isPerEvent()` if it is per event), a row in the seeder, and the call site.
Add an office screen	A view in `resources/js/admin/views`, a route with `meta.ability` in `admin/router.ts`, an entry in the `navigation` array in `AppLayout.vue`, and a controller under `Api/V1/Admin`.
Add a customer portal screen	Same, but `resources/js/portal` and the `/my` children in `admin/router.ts`. It ships inside the admin bundle — see section 09.
Add a public page	`resources/js/site/views` + `site/router.ts`. Check the exclusion list in `routes/web.php` if the path collides.
Change the home banner	Banners are records, not code: `Banner` model, managed in the office. Layout is `resources/js/site/BannerSlider.vue`.
Change SEO	Settings screen. Rendered server-side by `resources/views/site.blade.php` — it must be server-side or a crawler will not see it.
Change the policy pages	Settings screen, rich text. Shipped wording lives in `app/Support/Settings/PolicyText.php`.
Change retention	`app/Console/Commands/PruneServiceHistory.php — currently fifty days, as the requirements document asks.`
Change a scheduled time	`routes/console.php. Read the comments first; the order is deliberate.`
Add a sortable column	`app/Support/Http/SortsLists.php` — sorting is whitelisted, so an unlisted column is ignored rather than injected.
Add a master type	`app/Support/Masters/MasterRegistry.php — the Masters screen builds itself from it.`

SECTION 09

The front end

9.1

Two bundles

vite.config.ts — builds `site/main.ts` and `admin/main.ts` separately

A visitor reading the price list does not download the reports screen or the masters form. The **portal lives inside the admin bundle** because it needs the session and the auth store; it has its own layout and its own routes, and the router keeps the two apart by role, not by ability:

```
if (auth.isCustomer !== (to.meta.customer === true)) return home;
```

Three URL spaces, one blade decision

routes/web.php

```
Route::view('/login',        'app')->name('login.page');
Route::view('/app/{any?}',   'app')->where('any', '.*')->name('app');
Route::view('/my/{any?}',    'app')->where('any', '.*')->name('portal');

Route::view('/{any?}', 'site')
    ->where('any', '^(?!api|sanctum|up|storage|login|app|my).*')
    ->name('site');
```

THE TRAP

Any URL not in that exclusion list falls through to the **public** bundle, whose catch-all route redirects to the home page. When `/my` was missing here, every reload of a customer's page bounced them to the public home page and it looked like a session bug. If you add a URL space, add it to both the `Route::view` list *and* the negative lookahead.

9.2

Where logic goes

Not in `.vue` files. Anything with a decision in it — the signup flow, row selection, the subscriptions API — is a `.ts` module beside the views that use it (`site/signup.ts`, `admin/shared/useRowSelection.ts`, `admin/shared/subscriptions.api.ts`, `portal/portal.api.ts`, `shared/api/checkout.ts`). A component binds and renders; it does not decide.

This is applied to new work and to components as they are touched, not retrospectively across all of them. A bulk extraction was attempted and reverted; it broke 190 type checks in one go. Do it one component at a time, with the type check green between each.

Data fetching

TanStack Query throughout, with the query key naming the thing and its filters. Mutations invalidate rather than patching caches by hand. Axios lives in `shared/api/client.ts` and carries the Sanctum cookie.

Theme

One set of CSS custom properties in `resources/css/app.css`, light and dark, shared by both bundles. Use the token classes (`bg-surface`, `text-ink`, `border-line`, `text-accent`) rather than raw Tailwind colours, or the component will be wrong in one of the two themes.

Type checking

```
npx vue-tsc --noEmit
```

Run this rather than `npm run build` while working — `npm run dev` is normally already running and a build fights it.

SECTION 10

Conventions

10.1

RULE	DETAIL
UUIDv7 keys	`BaseModel` brings `HasUuids`. Time-ordered, so they still index well, and an id in a URL reveals nothing about how many customers there are.
Integer paise	Columns end `_paise`. No floats, no decimals, formatted only at the edge.
Soft deletes	On `BaseModel`. Deleting hides; history stays joinable.
Audit columns	`HasAuditColumns` fills created-by and updated-by from the session.
Derived, not stored	"Expired" is a date comparison, not a column somebody has to remember to update. Same for the order type and the plan's standing.
Append-only ledgers	The cloth ledger is never edited. A correction is a new entry, and the weekly check proves the cached balance still matches the sum.
Sanitise on write	`Support/Content/HtmlSanitizer`, at the point of storing. Sanitising on read leaves the mess in the table for every future reader to remember to clean, and one that forgets is a hole.
Whitelist, do not blacklist	Settings keys, sortable columns and master types are all allow-lists. Anything not on the list is dropped, not stored.
Enums for state	Backed enums in `app/Enums`, cast on the model. No magic strings in controllers.
Comments say why	The codebase is heavily commented and the comments explain reasoning, not mechanics. When you change behaviour a comment describes, change the comment in the same edit.

SECTION 11

Tests

11.1

```
php artisan test                                    # all 395
php artisan test --filter=AddSecondCarTest          # one file
npx vue-tsc --noEmit                                # the front end
```

Why the suite is fast

> **tests/Concerns/MigratesOnlyWhenStale.php**

It hooks `setUpTraits()` and, when every migration on disk is already recorded in the test database, sets `RefreshDatabaseState::$migrated = true` so `migrate:fresh` is skipped. That took the suite from 217 seconds to roughly forty-seven. Add a migration and the next run rebuilds automatically — there is nothing to remember.

> *Both* a schema dump and paratest were tried before this and made the suite *slower* — nine minutes. They are not the answer; do not reintroduce them.

The two tests that guard the rest

TEST	CATCHES
`Support/ClassNamesResolveTest`	Walks `app/` and fails if any referenced class name will not resolve. It exists because a missing `use Log;` made complaint creation return 500 *after* successfully creating the complaint.
`Api/MasterListingTest`	Walks `MasterRegistry` and opens every master, with and without filters. A master added later is covered the day it is added, without anybody remembering to come back.
`BranchScopeTest`	That the tenancy scope is still fail-closed.

Writing one

- `SectorContext::reset()` in both `setUp` and `tearDown`. Leaked branch state makes an unrelated test fail later and somewhere else.
- `fromSpa()` when the route needs the Origin header.
- Assert reads through `SectorContext::withoutScope()`, or the scope will hide the row you just created.
- Any paying route gets a "price is not taken from the request" test.

What has no automated cover yet

The cleaner app, the daily round and the reminder jobs. Worth knowing before you change any of them.

SECTION 12

Commands and the schedule

12.1

COMMAND	DOES	SCHEDULED
`eswachh:backup`	Dumps the database, verifies it, prunes old copies. A failure raises a critical alert, not just a log line.	00:10, keep 14
`eswachh:reconcile-payments`	Settles attempts that never came back from the gateway.	00:20, `--days=7`
`eswachh:prune-service-history`	Deletes cleaning records past the retention period.	00:40
`eswachh:send-renewal-reminders`	Messages customers due or overdue.	09:30
`eswachh:hold-overdue`	Puts long-overdue plans on hold.	10:00, `--grace=7`
`eswachh:check-cloth-balances`	Verifies cached balances against the ledger. Exits non-zero on a mismatch.	Mon 06:00
`eswachh:check-integrations`	Reports whether payments and messaging are configured, and what is stopping them. `--ping` proves the key.	By hand
`eswachh:import`	Imports from the v1 database.	By hand
`eswachh:backfill-invoice-numbers`	Issues invoice numbers for captured payments that have none.	Called by the import

> **THE ORDERING IS A DESIGN DECISION**
>
> Backup first, so a later job going wrong leaves a copy from before it did. Settlement before reminders, so nobody is chased for money they already paid. Holds last, so they act on a corrected picture. Pruning after the backup, so a retention rule can never be the reason something is unrecoverable.
>
> Every job is `withoutOverlapping`, `onOneServer` and `runInBackground`. Keep all three on anything you add: an unattended job that runs twice is worse than one that does not run at all.

SECTION 13

Bugs this codebase has already had

13.1 Each of these cost real time to find. They are recorded because every one of them is easy to reintroduce.

The timezone

v1 ran on `Asia/Kolkata`; v2 started on UTC. Every imported timestamp was mislabelled by five and a half hours, so date filters found nothing and nobody could see why. `APP_TIMEZONE=Asia/Kolkata` is not a preference.

Sign-in codes that never expired

`login_codes.expires_at` was a MySQL `TIMESTAMP`, which carries an implicit `ON UPDATE CURRENT_TIMESTAMP`. Every wrong guess touched the row and pushed the expiry forward, so a code lived forever. Migration `0001_01_01_000022` changes it to `DATETIME` and spends the outstanding ones. **Never use `TIMESTAMP` for an expiry.**

Two hundred customers in the wrong books

A sector was pointed at another franchise and nothing anybody could see changed. Every record carried its own `branch_id` — a copy of where the customer lived — and that copy was what the scope read, so the picker appeared to work while two hundred customers stayed put and the new owner signed in to a screen of zeroes.

A cache of a derived fact will drift, and nothing was comparing the two. The fix was to stop storing it: territory is now worked out from `customers.sector_id` against `user_sector` on every query. See section 03.

> **POWERSHELL WILL CORRUPT YOUR FILES**
>
> Two ways, both silent. `Set-Content -Encoding utf8` on 5.1 writes a **BOM**, and PHP will not parse a file that starts with one — 62 files at once. And a `-replace` round-trip through `ReadAllText` / `WriteAllText` mangled every non-ASCII character in eight more, turning `…` into mojibake that a second "repair" pass made worse.
>
> Use `sed`, which is byte-oriented, or the editor. If a file is already mangled, restore it from git rather than trying to decode it back.

Every master list at once

A variable that the listing closure never captured — `use ($definition, $filters)` where the body also wanted `$request`. One line, and it took out all twenty masters simultaneously: the price list, the geography, the banners, the templates. The screen said "Something went wrong" and the reason was only in the log.

PHP closures capture explicitly, so this class of mistake is silent until the line runs. It shipped because the Masters screen had *no test at all* — the single most load-bearing screen in the system, and nothing opened it. `MasterListingTest` now walks the registry.

The missing use statement

A complaint was created and then the notification's own catch block threw, because `Log` was not imported. The customer saw a 500 and the complaint existed. `ClassNamesResolveTest` now walks `app/` and fails on any name that will not resolve.

Every portal reload going home

`/my/*` was not listed in `routes/web.php`, so it fell through to the public bundle whose catch-all redirects home. Covered in section 09; it was reported twice as a session bug before the cause was found.

A cleared setting reverting to its default

`SiteSettings::read()` used `??`, which cannot tell a stored `null` from a missing row. Emptying a policy page and saving brought the shipped wording straight back, and looked like the save had failed. It uses `array_key_exists` now.

A settings cache that hid new settings

The SEO fields were added and the site kept serving the fallback title until somebody ran `cache:clear` by hand. The cache now rebuilds itself when the number of definitions changes.

Imported payments with no invoice number

All 492 of them. `eswachh:backfill-invoice-numbers` exists for this and the importer calls it.

A dropdown clipped by its own container

The plan list's action menu was cut off by an `overflow-x-auto` ancestor. It is teleported to `<body>` with fixed positioning and repositioned on scroll and resize. If you add a menu inside a scrolling table, do the same.

13.2

And one about tooling, not the app

> **`DO NOT EDIT PHP THROUGH SHELL HEREDOCS OR NODE SCRIPTS`**
>
> Passing PHP through a shell strips `$` sigils and `\` namespace separators, silently producing `AppModelsPayment`, `->actingAs(->owner)` and similar. This has caused real bugs in this repository, including the missing-import one above. Edit files with an editor.

House rules for this repository

- **Do not commit.** The owner commits manually.
- **Do not run** `npm run build` during development — `npm run dev` is normally already running. Use `npx vue-tsc --noEmit`.
- **Finish the whole task, then run** `php artisan test` **once.** Not after each step.

Still open

- Cloth count selectable while renewing on the *public* renew page.

- Pending renewals and cloths-below-five as dedicated pages rather than saved filters.
- Society-wise WhatsApp targeting could be more direct than it is.
- An "are you sure?" confirmation before a cleaner marks a car cleaned.
- No automated tests for the cleaner app, the daily round or the reminders.
- Five v1 orders (₹3,045) cannot be imported; their customers were deleted in v1.
- Extracting logic out of the remaining `.vue` components — parked by the owner, one of twenty-one done.